

B.TECH CSE — JAVA PROGRAMMING CASE STUDY

Movie Ticket Booking System

A Java Swing Desktop Application

Submitted By

Jaikishan Suthar

Roll No: **150096752152**

B.Tech Computer Science & Engineering

Java 17+

Java Swing

OOP

Collections Framework

Exception Handling

CRUD

Table of Contents

1	Case Study Introduction	3
2	Technologies & Java Features Used	3
3	Project Structure	4
4	Java File Explanation	4
5	OOP Concepts Used	6
6	Java Collections Used	7
7	GUI Components Used	8
8	Project Working Flow	9
9	Screenshots	9
10	Conclusion	10

1 Case Study Introduction

What is the Movie Ticket Booking System?

The Movie Ticket Booking System is a desktop application built using Java and Java Swing. It allows a cinema to manage movies, show timings, seats, and ticket bookings — all from a single window on a computer.

What Problem Does It Solve?

Earlier, booking a movie ticket required standing in a long queue at the cinema counter. This application removes that problem by providing a simple digital system where the operator can add movies, create show timings, let customers select their seat, book a ticket, and get a printed bill — all in a few clicks.

What Have I Created?

Movie Management

Add, update, search and delete movies with details like name, genre, duration, and rating.

Show Management

Create show timings for any movie. Shows are sorted automatically by time using TreeMap.

Seat Booking

View available seats (A1–C5 layout), select a seat, enter customer details, and confirm booking.

Ticket & Billing

Every booking generates a unique Ticket ID with a formatted bill showing price ₹200.

Why Is It Useful?

This system demonstrates how Java's core features — Collections, OOP, Swing GUI, and Exception Handling — work together to build a complete, working desktop application. It is useful for a college lab, a small cinema, or as a practical demonstration of real-world Java programming concepts.

2 Technologies & Java Features Used

Technology / Feature	What It Is	How I Used It
Java 17+	Core programming language	Entire application is written in Java
Java Swing	Built-in Java GUI library	JFrame, JPanel, JTable, JButton, JComboBox, JOptionPane — all from Swing
OOP (Object-Oriented Programming)	Class, Object, Encapsulation, Inheritance	Movie, Show, Customer, Ticket are separate classes with private fields and public methods
Java Collections Framework	Ready-made data structures	ArrayList, LinkedList, HashMap, TreeMap — used in BookingSystem.java
Array	Fixed-size collection	<code>String[] seats</code> and <code>boolean[] seatBooked</code> in Show.java
Exception Handling	try-catch, custom exceptions	SeatAlreadyBookedException, validation errors caught with try-catch
Validation	Input checking before processing	Phone number (10 digits), empty name, empty movie name — checked before saving
CRUD Operations	Create, Read, Update, Delete	Full CRUD for Movie; Create + Read for Customer, Show, Ticket
Searching	Finding specific records	Movie search by name, Ticket search by ID (HashMap), Show search by ID
Sorting	Ordering records	<code>Collections.sort()</code> with <code>Comparator</code> — sort movies by name or rating
Event Handling	Responding to button clicks	<code>ActionListener</code> on every button in MoviePanel, BookingPanel, TicketPanel

3 Project Structure

```
MovieTicketBookingSystem/
|
├── src/
|   ├── model/
|   |   ├── Movie.java           ← Stores movie details
|   |   ├── Show.java           ← Stores show + seat data
|   |   ├── Customer.java       ← Stores customer info
|   |   ├── Ticket.java         ← Stores booking + bill
|   |   └── SeatAlreadyBookedException.java ← Custom exception
|   ├── service/
|   |   └── BookingSystem.java   ← All logic + all collections
|   ├── gui/
|   |   ├── MainFrame.java      ← Main window + tabs
|   |   ├── MoviePanel.java     ← Movie CRUD tab
|   |   ├── BookingPanel.java   ← Book Ticket tab
|   |   └── TicketPanel.java    ← My Tickets tab
|   └── Main.java               ← Entry point
├── README.md
└── .gitignore
```

Total Java files: 11 (within the 8-12 student limit)

Quick Reference Table

File Name	Class Name	Package	Purpose
Main.java	Main	default	Application entry point — starts the Swing GUI on the Event Dispatch Thread
Movie.java	Movie	model	Stores movied, movieName, genre, duration, rating
Show.java	Show	model	Stores show details + seat array; handles bookSeat / cancelSeat
Customer.java	Customer	model	Stores customer info with static phone/name validation methods
Ticket.java	Ticket	model	Stores booking details; generates a formatted bill string
SeatAlreadyBookedException.java	SeatAlreadyBookedException	model	Custom checked exception thrown when a booked seat is selected again
BookingSystem.java	BookingSystem	service	Central service — holds all 5 collections + all business methods
MainFrame.java	MainFrame	gui	JFrame with 3 tabs + header + Exit button; creates shared BookingSystem
MoviePanel.java	MoviePanel	gui	Movies tab — Add/Update/Delete/Search/Sort/AddShow buttons + JTable
BookingPanel.java	BookingPanel	gui	Book Ticket tab — movie/show dropdowns, customer form, seat view, book
TicketPanel.java	TicketPanel	gui	My Tickets tab — search, cancel, view bill for any ticket

Main.java

Purpose: This is the starting point of the application. It uses `SwingUtilities.invokeLater()` to launch `MainFrame` on the Swing Event Dispatch Thread (EDT) — the correct and safe way to start a Swing app.

```
public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MainFrame());
    }
}
```

Movie.java

Important variables: `movieId`, `movieName`, `genre`, `duration` (int), `rating` (double).

Important methods: constructor, getters (`getMovieId()` etc.), setters (`setMovieName()` etc.), `toString()` returning a pipe-separated line.

Viva line: "Movie.java is a model class that stores the details of one movie using private fields and provides public getter and setter methods — this is encapsulation."

Show.java

Important variables: `String[] seats` (15 fixed seat names A1–C5), `boolean[] seatBooked` (tracks which seats are booked), `Movie movie`, `String showTime`.

Important methods: `isSeatAvailable()`, `bookSeat()` (throws Exception if already booked), `cancelSeat()`, `getAllSeatsInfo()`.

Viva line: "Show.java uses two parallel arrays — `seats[]` for seat names and `seatBooked[]` for their availability — to track the cinema seating layout."

Customer.java

Important variables: `customerId`, `name`, `phone`, `email`.

Important methods: `isValidPhone(String)` — static method using regex `\d{10}` to verify 10-digit phone; `isValidName(String)` — checks non-empty.

Viva line: "Customer.java includes static validation methods so I can validate input before creating a Customer object."

Ticket.java

Important variables: `ticketId`, `Customer customer`, `Movie movie`, `Show show`, `seatNumber`, `price` (200.0).

Important methods: `getBillInfo()` — returns a formatted multi-line bill string; `toString()` — single-line summary.

Viva line: "Ticket.java connects Customer, Movie, and Show objects together, showing object composition in OOP."

SeatAlreadyBookedException.java

Purpose: A custom checked exception that extends `Exception`. It is thrown specifically when a user tries to book a seat that is already taken, giving a meaningful, specific error message instead of a generic one.

Viva line: "SeatAlreadyBookedException extends Exception — this is inheritance. I call `super(message)` to pass the error message to the parent Exception class."

BookingSystem.java

Purpose: This is the service class that holds all the data (all 5 collections) and all the application logic. GUI panels do not do any logic themselves — they call `BookingSystem` methods.

Important variables: `ArrayList<Movie> movies`, `ArrayList<Customer> customers`, `LinkedList<Ticket> bookingList`, `HashMap<String, Ticket> tickets`, `TreeMap<String, Show> shows`.

Important methods: `addMovie()`, `updateMovie()`, `deleteMovie()`, `searchMovie()`, `addCustomer()`, `addShow()`, `bookTicket()`, `cancelTicket()`, `searchTicket()`, `getMoviesSortedByName()`, `getMoviesSortedByRating()`.

Viva line: "BookingSystem.java is my service class — it is the brain of the application. All collections and all methods live here."

MainFrame.java

Purpose: Creates the main `JFrame` window (900×600). Adds a blue header label, a `JTabbedPane` with 3 tabs (Movies, Book Ticket, My Tickets), and a red Exit button in the footer. Creates one shared `BookingSystem` object and passes it to all three panels.

Viva line: "MainFrame extends `JFrame` — it is the main window. I pass one `BookingSystem` object to all panels so they share the same data."

MoviePanel.java

Purpose: The Movies tab. Lets the user add, update, delete, search, and sort movies. Also has an "Add Show for Movie" button. Displays movies in a `JTable`. Clicking a row auto-fills the Movie ID field using a `ListSelectionListener`.

Viva line: "MoviePanel contains the full CRUD for movies — Add, Update, Delete, and Search — all through buttons that call `BookingSystem` methods."

BookingPanel.java

Purpose: The Book Ticket tab. Has a `JComboBox` for movies, a `JComboBox` for shows (loaded dynamically when movie changes via `ActionListener`), text fields for customer details, and a seat view area. Calls `bookingSystem.bookTicket()` and shows the bill in `JOptionPane` on success.

Viva line: "BookingPanel shows the seat map and calls `bookTicket()` — it handles both `SeatAlreadyBookedException` and generic Exception separately with two catch blocks."

TicketPanel.java

Purpose: The My Tickets tab. Shows all booked tickets in a JTable (iterates over LinkedList). Allows search by Ticket ID (HashMap lookup), cancel ticket (frees the seat), and View Bill (shows formatted bill in JOptionPane).

Viva line: "TicketPanel uses the LinkedList to display all tickets in booking order, and the HashMap to do fast search and cancellation by Ticket ID."

5 OOP Concepts Used

OOP Concept	Used?	File / Class	Example from My Project
Class	Used	All 11 files	Every file defines a class: <code>class Movie</code> , <code>class BookingSystem</code> , etc.
Object	Used	BookingSystem.java, MainFrame.java	<code>new Movie(...)</code> , <code>new Show(...)</code> , <code>new Ticket(...)</code> , <code>new BookingSystem()</code> — objects created throughout
Encapsulation	Used	Movie.java, Customer.java, Show.java, Ticket.java	All fields are <code>private</code> . External access only through <code>public</code> getters/setters (e.g. <code>getMovieName()</code> , <code>setMovieName()</code>)
Constructor	Used	All model classes	<code>public Movie(String movieId, String movieName, String genre, int duration, double rating)</code> — sets all fields on object creation
Inheritance	Used	SeatAlreadyBookedException.java, MainFrame.java, all Panel classes	<code>SeatAlreadyBookedException extends Exception</code> ; <code>MainFrame extends JFrame</code> ; <code>MoviePanel extends JPanel</code>
Method Overriding	Used	All model classes	<code>@Override toString()</code> in Movie, Show, Customer, Ticket — overrides Object's default toString
Abstraction	Not directly	—	No abstract class or interface defined. BookingSystem provides logical abstraction (hides logic from GUI) but no Java <code>abstract</code> keyword is used.
Polymorphism	Not directly	—	No method overloading or interface polymorphism. Exception catching uses Java's built-in polymorphic dispatch (<code>catch(Exception e)</code>).

6 Java Collections Used

All five collections are declared and used in **BookingSystem.java**. They are easy to find — they are the first five field declarations at the top of that class.

Collection	Variable Name (exact)	What It Stores	Why This Type
Array	<code>String[] seats</code> <code>boolean[] seatBooked</code>	Fixed 15 seat names (A1–C5) and their booked/free status	Seat count is fixed (15), so a fixed-size array is the right choice. Used in Show.java
ArrayList<Movie>	<code>movies</code>	All movies in the system	Dynamic size — movies can be added/deleted. Supports index-based access. Used in <code>addMovie()</code> , <code>deleteMovie()</code> , <code>getMovies()</code>
ArrayList<Customer>	<code>customers</code>	All registered customers	Same reason as movies — grows dynamically. Used in <code>addCustomer()</code> , <code>getCustomers()</code>
LinkedList<Ticket>	<code>bookingList</code>	All booked tickets in booking order	Preserves insertion order — first booked appears first. Efficient add/remove. Used in <code>bookTicket()</code> , <code>cancelTicket()</code> , <code>getBookingList()</code>
HashMap<String, Ticket>	<code>tickets</code>	Tickets with Ticket ID as key (T001, T002, ...)	O(1) lookup — <code>tickets.get("T001")</code> finds the ticket instantly without looping. Used in <code>searchTicket()</code> , <code>cancelTicket()</code>
TreeMap<String, Show>	<code>shows</code>	All shows, key = <code>showTime + "_" + showId</code>	Automatically keeps shows sorted by time — no extra sort code needed. Used in <code>addShow()</code> , <code>getShows()</code> , <code>getShowsForMovie()</code>

Where to Point in the Code (for Viva)

```
// BookingSystem.java – top of the class – show these 5 lines:
private ArrayList<Movie>      movies      = new ArrayList<>();
private ArrayList<Customer>   customers   = new ArrayList<>();
private LinkedList<Ticket>    bookingList = new LinkedList<>();
private HashMap<String, Ticket> tickets    = new HashMap<>();
private TreeMap<String, Show> shows      = new TreeMap<>();
```

Sorting with Collections.sort() and Comparator

```
// BookingSystem.java – getMoviesSortedByName()
Collections.sort(sorted, Comparator.comparing(Movie::getMovieName));

// BookingSystem.java – getMoviesSortedByRating() (highest first)
sorted.sort((m1, m2) -> Double.compare(m2.getRating(), m1.getRating()));
```

7 GUI Components Used (Java Swing)

Component	What It Does	Where Used in My Project
<code>JFrame</code>	The main window of the application	<code>MainFrame</code> extends <code>JFrame</code> — the 900×600 main window
<code>JPanel</code>	A container that groups other components	Used in all GUI classes for form panels, button panels, seat panels
<code>JTabbedPane</code>	Shows multiple tabs in one window	<code>MainFrame.java</code> — Movies / Book Ticket / My Tickets tabs
<code>JLabel</code>	Displays text on screen	All panels — field labels like "Movie Name:", "Select Show:", header text
<code>TextField</code>	Single-line text input box	MoviePanel (name, genre, duration, rating), BookingPanel (customer name, phone, email, seat)
<code>JButton</code>	Clickable button	All panels — Add Movie, Book Ticket, Cancel, Search, Exit, etc.
<code>JComboBox</code>	Dropdown selection	<code>BookingPanel.java</code> — <code>cmbMovie</code> (select movie) and <code>cmbShow</code> (select show)
<code>JTable</code>	Displays data in rows and columns	<code>MoviePanel.java</code> (movie list), <code>TicketPanel.java</code> (ticket list)
<code>DefaultTableModel</code>	Controls data in JTable	MoviePanel and TicketPanel — <code>tableModel.addRow()</code> , <code>setRowCount(0)</code>
<code>JScrollPane</code>	Adds scroll bars to a component	Wraps JTable in MoviePanel and TicketPanel; wraps seat text area in BookingPanel
<code>JTextArea</code>	Multi-line text display	<code>BookingPanel.java</code> — <code>txtAvailableSeats</code> shows the seat map
<code>JOptionPane</code>	Pop-up dialog boxes	Success/error messages, bill display, delete confirmation, exit confirmation — everywhere
<code>BorderLayout</code>	Positions items: NORTH/CENTER/SOUTH	MainFrame, MoviePanel, BookingPanel, TicketPanel — outer layout
<code>GridLayout</code>	Equal-sized grid of cells	Form panels in MoviePanel (3×4 grid) and BookingPanel (8×2 grid)
<code>FlowLayout</code>	Arranges items left-to-right in a row	Button panels in all three GUI classes
<code>ActionListener</code>	Responds to button clicks / dropdown changes	Every button uses <code>btnAdd.addActionListener(e -> addMovie())</code> ; <code>cmbMovie</code> uses it to reload shows
<code>ListSelectionListener</code>	Responds to row selection in JTable	<code>MoviePanel.java</code> — clicking a movie row auto-fills the Movie ID field
<code>ChangeListener</code>	Responds to tab switching	<code>MainFrame.java</code> — refreshes panel data when user switches tabs

8 Project Working Flow

Main.java → MainFrame → BookingSystem created → Sample data loaded → 3 Tabs visible

Step-by-Step Flow

Step	What Happens	Class / Method Involved
1. App Start	Main.java calls new MainFrame() via SwingUtilities	Main.java → MainFrame.java
2. GUI Setup	MainFrame creates BookingSystem, creates 3 panels, adds them to tabs	MainFrame → BookingSystem, MoviePanel, BookingPanel, TicketPanel
3. Sample Data	4 movies (Avengers, Avatar, Inception, Interstellar) and 4 shows are added automatically	BookingSystem.loadSampleData()
4. Add Movie	User fills form → clicks Add Movie → BookingSystem.addMovie() → ArrayList grows → table refreshes	MoviePanel → BookingSystem.addMovie()
5. Add Show	User selects a movie row → clicks "Add Show for Movie" → enters time → show added to TreeMap	MoviePanel.addShowForMovie() → BookingSystem.addShow()
6. Book Ticket	User selects movie + show → enters name/phone/email/seat → clicks Book Ticket	BookingPanel.bookTicket() → BookingSystem.bookTicket()
7. Validation	Empty name / invalid phone / booked seat → exception thrown → JOptionPane error shown	BookingSystem.bookTicket() → SeatAlreadyBookedException / Exception
8. Ticket Created	New Ticket object → stored in HashMap (fast lookup) + LinkedList (order) → bill shown in popup	Ticket.java, tickets.put(), bookingList.add()
9. View Tickets	My Tickets tab → refreshTable() iterates LinkedList → fills JTable	TicketPanel.refreshTable() → bookingSystem.getBookingList()
10. Search Ticket	Enter Ticket ID → searchTicket() → HashMap.get() → O(1) lookup → show in table	TicketPanel → BookingSystem.searchTicket() → tickets.get()
11. Cancel Ticket	Ticket found → show.cancelSeat() frees seat → removed from HashMap + LinkedList	BookingSystem.cancelTicket()
12. View Bill	Ticket.getBillInfo() returns formatted string → shown in JOptionPane	Ticket.getBillInfo() → JOptionPane.showMessageDialog()

Application Overview

[Screenshot not available: Screenshot 2026-09-29 at 10.44.55 AM.png]

What the Application Shows

Tab / Screen	What You See	Key Features Visible
Movies Tab	List of all movies in a JTable with columns: Movie ID, Movie Name, Genre, Duration, Rating	Add Movie, Update, Delete, Search, Add Show for Movie, Sort by Name/Rating, Refresh — 8 colour-coded buttons
Book Ticket Tab	Left side: booking form (movie dropdown, show dropdown, customer fields, seat input). Right side: seat map showing A1–C5 grid	Shows booked seats as [A1]; available seats as A1. Three buttons: Book Ticket (green), Clear (grey), Refresh Movies (blue)
My Tickets Tab	Table of all booked tickets with columns: Ticket ID, Customer, Movie, Show Time, Seat, Price	Search by Ticket ID, Cancel Ticket, View Bill — bill shown in a popup with formatted layout
Bill Popup	JOptionPane showing the full bill: Ticket ID, Customer name, Movie, Show Time, Seat, Price ₹200, Total	Generated by Ticket.getBillInfo() method

10

Conclusion

The Movie Ticket Booking System is a complete Java desktop application built as a B.Tech CSE Java Programming case study. The project successfully demonstrates all the required Java concepts in a simple, working, and easy-to-understand way.

What I Learned

Concept	What I Applied
OOP	Designed separate classes (Movie, Show, Customer, Ticket) with encapsulation, constructors, and method overriding
Collections Framework	Used 5 different collections — each chosen for a specific reason (ArrayList for movies, HashMap for fast ticket lookup, TreeMap for sorted shows)
Java Swing	Built a fully working GUI with JFrame, JTabbedPane, JTable, JComboBox, JOptionPane — no external library needed
Exception Handling	Created a custom exception (SeatAlreadyBookedException) and used try-catch blocks with meaningful error messages
CRUD + Search + Sort	Full CRUD for movies; search by ID and name; sort by name and rating using Collections.sort() and Comparator
Validation	Validated phone (10 digits regex), empty names, seat existence, duplicate booking — all before any data is saved

Through this project I gained practical experience of how Java's core features work together to build a real application — not just in theory, but as a complete, runnable program that can be demonstrated and explained step by step.